

# **FIRECAR**

Project Team

|                |          |
|----------------|----------|
| Alyan Shahid   | 21I-1537 |
| Muhammad Zafar | 21I-1534 |
| Ahmed Awaiz    | 21I-1539 |

Session 2021-2025

Supervised by

**Dr. Zeshan Khan**



**Department of Computer Science**

**National University of Computer and Emerging Sciences  
Islamabad, Pakistan**

**October, 2024**

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>1</b>  |
| 1.1      | Problem Statement . . . . .                           | 1         |
| 1.2      | Scope . . . . .                                       | 2         |
| 1.3      | Modules . . . . .                                     | 2         |
| 1.3.1    | Module 1 . . . . .                                    | 3         |
| 1.3.2    | Module 2 . . . . .                                    | 3         |
| 1.4      | User Classes and Characteristics . . . . .            | 4         |
| <b>2</b> | <b>Project Requirements</b>                           | <b>5</b>  |
| 2.1      | Use-case/Event Response Table/Storyboarding . . . . . | 5         |
| 2.1.1    | USE CASE DIAGRAM . . . . .                            | 5         |
| 2.1.2    | DETAILED USE CASES . . . . .                          | 6         |
| 2.1.3    | EVENT RESPONSE TABLE . . . . .                        | 14        |
| 2.1.4    | STORY BOARDING . . . . .                              | 15        |
| 2.2      | Functional Requirements . . . . .                     | 16        |
| 2.2.1    | Module 1 . . . . .                                    | 16        |
| 2.2.2    | Module 2 . . . . .                                    | 16        |
| 2.3      | Non-Functional Requirements . . . . .                 | 17        |
| 2.3.1    | Reliability . . . . .                                 | 17        |
| 2.3.2    | Usability . . . . .                                   | 17        |
| 2.3.3    | Performance . . . . .                                 | 17        |
| 2.3.4    | Security . . . . .                                    | 17        |
| 2.4      | Domain Model . . . . .                                | 18        |
| <b>3</b> | <b>System Overview</b>                                | <b>19</b> |
| 3.1      | Architectural Design . . . . .                        | 19        |
| 3.1.1    | Modular Structure and Relationships: . . . . .        | 19        |
| 3.1.2    | High-Level Collaboration: . . . . .                   | 20        |
| 3.2      | Design Models . . . . .                               | 22        |
| 3.3      | Data Design . . . . .                                 | 26        |
| 3.3.1    | Real-time Data Capture . . . . .                      | 26        |
| 3.3.2    | Data Structures . . . . .                             | 26        |
| 3.3.3    | Data Processing . . . . .                             | 26        |

|       |                                          |           |
|-------|------------------------------------------|-----------|
| 3.3.4 | Data Organization with MongoDB . . . . . | 26        |
|       | <b>References</b>                        | <b>29</b> |

# List of Figures

|     |                                        |    |
|-----|----------------------------------------|----|
| 2.1 | Use Case Diagram . . . . .             | 5  |
| 2.2 | Domain Model . . . . .                 | 18 |
| 3.1 | Activity Diagram . . . . .             | 22 |
| 3.2 | Class Diagram . . . . .                | 23 |
| 3.3 | Class Level Sequence Diagram . . . . . | 24 |
| 3.4 | State Transition Diagram . . . . .     | 25 |

# List of Tables

|     |                                            |    |
|-----|--------------------------------------------|----|
| 1.1 | User Classes and Characteristics . . . . . | 4  |
| 2.1 | Event Response Table . . . . .             | 14 |
| 2.2 | Story Boarding . . . . .                   | 15 |

# Chapter 1

## Introduction

FireCar is designed to serve as an autonomous indoor firefighting robot, especially for environments like server rooms and data centers where constant human supervision is not feasible. Its main activity is patrolling fixed areas for the occurrence of fire and taking proper steps to extinguish it before spreading. The system is a component-heavy model with sensors, motors, batteries, cameras and actuators being controlled by an embedded system and edge-AI. FireCar is a four-wheeled vehicle controlled by BO(Battery Operated) motors to move in the XY plane. It uses embedded software for room mapping, obstacle avoidance, fire detection, and communication controlled by a Raspberry Pi 4. It visualizes the environment using a Raspberry Pi camera, detecting hazardous materials and obstacles. Fire detection is done by computer vision with OpenCV, which automatically turns on the fire extinguishing system in case of flame or smoke detection. This robot has its movement enabled on four BO-motor-driven wheels, communicating over Wi-Fi to update the Web Application with fire detection and status. Additionally, it is mounted with a fire extinguisher that releases CO<sub>2</sub>/water mist to extinguish the source of fire directly.

### 1.1 Problem Statement

In emergency operations, speed and accuracy both play an important role. Traditional methods of fire response can sometimes face delays, and when you're working with something as destructive as fire, every second counts. Fire outbreaks within the sensitive environment of server rooms or data centers result in great losses not only regarding expensive equipment but even critical data. Because of huge consumption of electricity and generation of high heat, these places are extremely susceptible to fire hazards, and even small fires could be very destructive. The majority of the current fire detection systems are pretty slow in response; sprinklers, aside from being traditional, can have

destructive effects on costly electronics due to water exposure. It is very important to solve this problem of quick fire detection and control in these areas. The system is under development to introduce an efficient automated solution to the gap existing between fire detection and effective, localized fire suppression in these high-hazard areas. Thus, it saves time by automating the operation of fire detection and control, reducing human involvement hence reducing risk to life in hazardous conditions. The system contributes to solving the problem of over-damage created by conventional methods, such as sprinklers, through its targeted suppression. It reduces hazards presented by dangerous materials and challenges brought about by obstacles, making the environment much safer for valuable assets. This provides a more appropriate and safer way to control fire, matching the peculiar challenges posed by server rooms and data centers today.

## 1.2 Scope

The project encompasses the design and construction of a mobile robot that can detect and put out fires autonomously in indoor environments, as well as triggering alarms and reporting its state to a web application. The robot operates on the XY plane and will be able to safely traverse complex indoor implied spaces while avoiding obstacles and stairs. The robot will include fire, gas, and smoke detection sensors to detect fire hazards early on. Once a fire is detected, the robot will be able to safely autonomously navigate to the fire source using obstacle detection and avoidance algorithms, and then extinguish the fire using its fire extinguishing arm. In addition to extinguishing fires, the robot will also generate a map of the room using embedded sensors and cameras, assisting it with navigation and identifying hazardous items. The system will have real-time notification alerts that trigger alarms and alerts a web-based application for remote monitoring, alerting, and logging of incidents. It will also be integrated with existing fire alarm systems to provide a timely response in emergencies. The robot will need to be very vigilant and continuously monitor the environment for gas and smoke as an extra precaution in deadly situations. The robot is designed to operate autonomously allowing reduced human intervention and allowing barriers to be removed from dangerous situations. In summary, the project scope includes fire detection, autonomous navigation, obstacle avoidance, fire exhausting, and online monitoring alerts for comprehensive and complete fire protection in sensitive indoor environments.

## 1.3 Modules

Write down the modules of the proposed project. Each module should highlight features, using bulleted/numbered notation. When developing both a mobile app and a web app,

group the modules according to the system types, such as, Client Web App, Client Mobile App, Admin Web App etc.

### **1.3.1 Module 1**

This module includes camera-based room mapping, and obstacle identification. A camera generates a 2D room map which identifies obstacles for effective path planning. A simple user-friendly interface which uses the map, obstacles, and optimal paths the robot uses for navigation.

1. **Room Mapping:** The camera is mounted on the wall or ceiling, performing a scan to generate a 2D map of the area. It uses the feed from the camera to recognize obstacles and their locations thus providing clear vision around the room while planning paths for the robot to take around obstacles.
2. **Path Planning:** The system computes the best path for the camera to move for navigating around a room based on the information gathered from detected obstacles.
3. **User Interface:** A basic GUI showing the map of the room and the obstacles in that room along with all possible paths the robot may have. (Admin Web Application)

### **1.3.2 Module 2**

This module focuses on advanced fire detection and response mechanisms. Involves a combination of machine learning models for real-time fire detection and takes the robot to the fire location for suppression. System comprises a superior navigation system that avoids obstacles and a user interface for fire logs, manual control, and reporting.

1. **Fire Detection:** Implement fire detection by using advanced ML models and camera-based analysis. Accurately increase the precisions of real-time fire detection.
2. **Hardware Integration:** Develop the fully functional hardware of the FireCar.
3. **Better Navigation:** Advanced navigation algorithms that will enable the robot to make adjustments dynamically and travel to the position of the fire quickly by implementing reinforcement learning.
4. **User Interaction and Reporting:** The user interface needs to be improved with the input of fire logs, suppression actions, and manual control options.
5. **Driven Response:** System is guided automatically by the robot to the detected location of the fire for corrective action such as triggering of a mechanism which douses the fire through perhaps the CO2 system.

## 1.4 User Classes and Characteristics

| User class                | Description                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Admin/User                | The primary user responsible for monitoring the system's automatic operations and managing the optional manual control. |
| Building Maintenance Team | Users who ensure the building's safety and infrastructure, overseeing room mapping and gas detection capabilities.      |
| Emergency Response Team   | Users tasked with responding to fire or gas leak alerts generated by the MQ2 gas sensors and overhead camera data.      |
| Building Inhabitants      | General occupants of the building, concerned with safety and relying on the system's alerts for fire or gas leaks.      |

Table 1.1: User Classes and Characteristics

# Chapter 2

## Project Requirements

This chapter describes the functional and non-functional requirements of the project.

### 2.1 Use-case/Event Response Table/Storyboarding

#### 2.1.1 USE CASE DIAGRAM

The following is the Use Case Diagram.

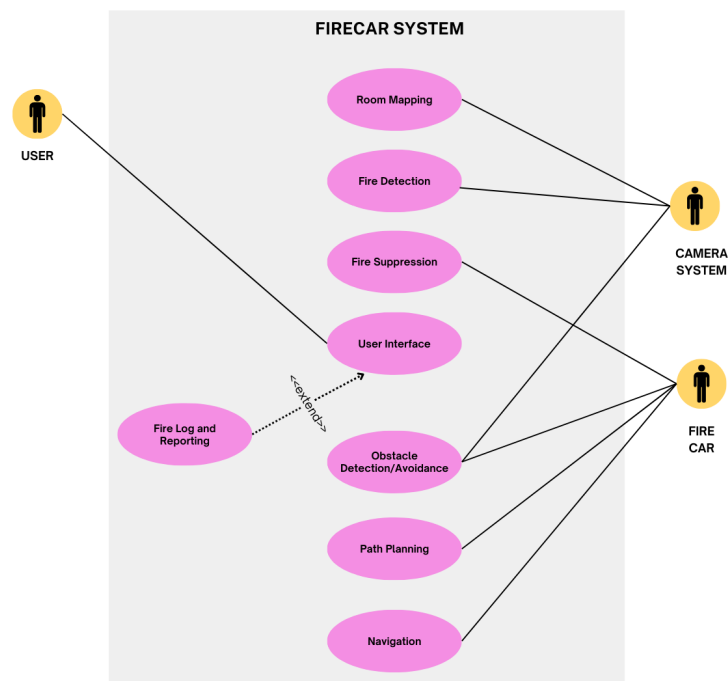


Figure 2.1: Use Case Diagram

## 2.1.2 DETAILED USE CASES

### Use Case 1: Room Mapping

Scope: FireCar System

Level: Subfunction level

Primary Actor: Camera System

Stakeholders and Interests:

- User: Interested in real-time mapping for fire detection and path planning.
- FireCar System: Requires the map for navigation and fire suppression.

Preconditions:

- The camera system is operational and has a clear line of sight.
- The FireCar system is powered on and awaiting map data.

Success Guarantee (Postcondition):

- A real-time, up-to-date room map is generated and sent to FireCar for navigation and path planning.

Main Success Scenario:

1. The camera system scans the room.
2. It identifies walls, obstacles, and open spaces.
3. A room map is created and transmitted to FireCar.
4. FireCar receives the map for path planning.

Extensions:

- If the camera is obstructed: The system retries scanning and logs an error for manual intervention if needed.

Special Requirements:

- Real-time updates with minimal delay.

Technology and Data Variations:

- Use of thermal or infrared cameras in low-visibility environments.

### **Use Case 2: Fire Detection**

Scope: FireCar System

Level: Subfunction level

Primary Actor: Camera System

Stakeholders and Interests:

- User: Wants early fire detection for safety.
- FireCar System: Needs the fire's location to respond.

Preconditions:

- Camera system and MQ2 sensors are monitoring the room.

Success Guarantee (Postcondition):

- Fire is detected, and location details are sent to FireCar and the User Interface.

Main Success Scenario:

1. Camera system scans for flames or heat signatures.
2. MQ2 sensors detect smoke or gas.
3. Fire or smoke is confirmed, and the location is sent to FireCar and the user.

Extensions:

- If fire detection fails: FireCar attempts manual confirmation or alerts the user for manual input.

### Special Requirements:

- Detection must happen within seconds.

### Technology and Data Variations:

- Different detection methods: flame detection, heat, smoke (MQ2 sensors).

### Use Case 3: Fire Suppression

Scope: FireCar System

Level: User goal level

Primary Actor: FireCar

Stakeholders and Interests:

- User: Wants the fire to be extinguished automatically.
- FireCar System: Aims to perform suppression autonomously.

### Preconditions:

- FireCar is at the fire location.
- Fire suppression mechanism is operational (e.g., CO2 or foam-based).

### Success Guarantee (Postcondition):

- Fire is successfully extinguished

### Main Success Scenario:

1. FireCar moves to the fire location.
2. Fire suppression system is activated.
3. Fire is extinguished, and status is sent to the user.

### Extensions:

- If suppression fails: FireCar escalates to manual control or external fire systems.

Special Requirements:

- System must suppress fires within seconds.

Technology and Data Variations:

- Use of multiple fire suppression mechanisms depending on fire type.

#### **Use Case 4: User Interface Notification**

Scope: FireCar System

Level: User goal level

Primary Actor: User

Stakeholders and Interests:

- User: Requires real-time updates about fire events.

Preconditions:

- User is connected to the system.

Success Guarantee (Postcondition):

- User is informed of fire detection, suppression, and current status.

Main Success Scenario:

1. Fire is detected by the camera or MQ2 sensors.
2. The user receives a notification with the fire's location.
3. The notification provides real-time updates on fire suppression status.

Extensions:

- If the user is offline: The system sends SMS or email notifications.

Special Requirements:

- Notifications should be real-time with no delays.

Technology and Data Variations:

- Web app and mobile app notifications.

### **Use Case 5: Fire Log and Reporting**

Scope: FireCar System

Level: Subfunction level

Primary Actor: User

Stakeholders and Interests:

- User: Requires historical data on fire incidents for analysis or reporting.

Preconditions:

- FireCar has managed fire incidents.

Success Guarantee (Postcondition):

- A comprehensive log and report of the fire event is produced.

Main Success Scenario:

1. FireCar documents the fire incident.
2. All pertinent data (detection time, location, suppression time, etc.) is recorded.
3. A report is created and made accessible to the user.

Extensions:

- If report generation fails: The system retries or alerts the user about the failure.

Special Requirements:

- Logs must be securely stored and accessible only to authorized users.

Technology and Data Variations:

- Reports can be exported in various formats (PDF, CSV, etc.).

### **Use Case 6: Obstacle Detection and Avoidance**

Scope: FireCar System

Level: Subfunction level

Primary actor: FireCar/Camera System

Stakeholders and interests:

- Users (residents) want to ensure safe navigation without collisions.

Preconditions:

- The overhead camera system is operational, and the room map has been generated.

Main Success Scenario:

1. FireCar continuously monitors the room map generated by the overhead cameras.
2. FireCar identifies any obstacles in its path.
3. FireCar recalculates its route to avoid the detected obstacles.
4. FireCar navigates around the obstacles and continues toward the fire location.

Extensions:

- If a significant obstacle is detected, FireCar may halt temporarily until it recalibrates its path.

Special requirements:

- The obstacle detection and avoidance system should operate in real-time and effectively adapt to dynamic environments.

Technology and data variations list:

- Overhead camera data processing, real-time obstacle detection algorithms, and path recalibration methods.

### **Use Case 7: Path Planning**

Scope: FireCar System

Level: Subfunction level

Primary Actor: FireCar

### Stakeholders and Interests:

- User: Wants a fast and efficient fire response.
- FireCar System: Aims to minimize response time.

### Preconditions:

- FireCar has received the room map and the location of the fire.

### Success Guarantee (Postcondition):

- FireCar reaches the fire using the most efficient route.

### Main Success Scenario:

1. FireCar obtains the room map and fire location.
2. FireCar employs reinforcement learning to determine the optimal path.
3. FireCar navigates the path while avoiding obstacles.

### Extensions:

- If the path is obstructed: FireCar recalculates a new route.

### Special Requirements:

- Path calculation must occur in real time

.

### Technology and Data Variations:

- Integration with reinforcement learning algorithms for optimal path planning.

## **Use Case 8: Navigation**

Scope: FireCar System

Level: User goal level

Primary actor: FireCar

Stakeholders and interests:

- Users (residents) want FireCar to reach the fire location quickly and safely.

Preconditions:

- The room map has been generated and updated by the overhead camera system.

Main Success Scenario:

1. FireCar receives the location of the detected fire from the Fire Detection system.
2. FireCar analyzes the room map generated by the overhead cameras.
3. FireCar identifies the best route to the fire location while considering obstacles.
4. FireCar moves autonomously toward the fire location.

Extensions:

- If a new obstacle is detected during navigation, FireCar re-evaluates its path based on the updated map and reroutes.

Special requirements:

- The navigation system should be efficient and responsive to real-time changes in the environment.

Technology and data variations list:

- Overhead camera data formats, mapping algorithms, and navigation protocols.

### 2.1.3 EVENT RESPONSE TABLE

| Event                        | Source                       | Response                                 | Action                                     |
|------------------------------|------------------------------|------------------------------------------|--------------------------------------------|
| Fire Detected                | Camera System / MQ2 Sensors  | Validate fire presence and location      | FireCar navigates to fire location         |
| Obstacle Detected (from map) | Overhead Camera System       | Recalculate path based on room map       | FireCar changes route to avoid obstacle    |
| Gas/Smoke Detected           | MQ2 Sensors                  | Issue alert                              | Notify user via User Interface             |
| Fire Suppressed              | FireCar Extinguishing System | Confirm suppression status and log event | Update User Interface with fire log        |
| Low Battery Warning          | Power Management System      | Stop operation, send notification        | FireCar returns to charging station        |
| FireCar at Fire Location     | Navigation System            | Activate fire suppression mechanism      | Begin extinguishing process                |
| Room Mapping Requested       | Backend System               | Scan room and generate updated map       | Send updated map to FireCar for navigation |
| Report Requested by User     | User Interface               | Fetch fire event logs                    | Present report via web app                 |
| Fire Not Detected            | Camera System / MQ2 Sensors  | Log false alarm and send report to user  | Notify user of false alarm                 |
| User Manual Override         | User Interface               | Stop autonomous operation                | Wait for user commands                     |

Table 2.1: Event Response Table

## 2.1.4 STORY BOARDING

| Scene                               | Description                                                                       | User Interaction                                          | Backend Process                                                        |
|-------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------|
| Scene 1:<br>Dashboard Home          | The main dashboard shows FireCar's current status (battery level, room map, etc.) | User views real-time data                                 | Backend fetches data from FireCar and sensors                          |
| Scene 2:<br>Fire Detected Alert     | A popup appears indicating a fire detected in Room A                              | User can click "View Details" to navigate to the fire     | Backend logs fire data and triggers FireCar navigation to the location |
| Scene 3:<br>FireCar Navigation View | User sees FireCar's live movement on the room map towards the fire                | User watches the FireCar's path on the map                | Backend provides real-time path data from FireCar                      |
| Scene 4:<br>Fire Suppression        | A video feed shows FireCar extinguishing the fire                                 | User can monitor or intervene manually via the interface  | Backend logs fire suppression data and updates the system              |
| Scene 5:<br>Report Generated        | After the fire is extinguished, a summary report is generated                     | User can download or review fire suppression reports      | Backend compiles logs and generates a fire report                      |
| Scene 6:<br>Manual Control          | If needed, user switches to manual control for FireCar navigation                 | User clicks "Take Control" and navigates FireCar manually | Backend switches from autonomous to manual mode                        |

Table 2.2: Story Boarding

## **2.2 Functional Requirements**

### **2.2.1 Module 1**

#### Camera-Based Room Mapping and Identifying Obstacles

1. FR1: The system will create a 2D map of the room using input from the camera and recognize all obstacles present.
2. FR2: The system will refresh the room map in real-time as it identifies new obstacles.
3. FR3: The system will actively steer clear of obstacles while moving through the room based on their detected locations.
4. FR4: The system will show the room map, including obstacles, on the user interface for easy monitoring and control.
5. FR5: The system shall compute and follow the optimal path for navigation based on real-time map updates.

### **2.2.2 Module 2**

#### Advanced mechanisms for fire detection and response

1. FR1: The system will detect fire and smoke in real-time by utilizing the camera and sophisticated machine learning models.
2. FR2: The system will automatically navigate to the fire's location, avoiding obstacles, and activate the fire suppression mechanism (either CO2 or water mist).
3. FR3: The system will create a log entry for each fire detection and suppression event, detailing the time, location, and the suppression method used.
4. FR4: The system will enable the admin to manually control the fire suppression mechanism in special circumstances.
5. FR5: The system will offer the admin the ability to review logs and fire events through the user interface.

## **2.3 Non-Functional Requirements**

### **2.3.1 Reliability**

1. REL-1: The system must achieve a Mean Time Between Failures (MTBF) of 1000 hours when operating in a stable environment.
2. REL-2: In the event of a failure in the camera or fire detection sensors, the system will alert the administrator and stop operations to prevent any errors.

### **2.3.2 Usability**

1. USE-1: Users should be able to access the room map and see obstacle locations within three clicks on the admin interface.
2. USE-2: The admin must have the ability to manually operate the fire suppression system through an intuitive interface in just two clicks.
3. USE-3: Fire event logs should be easily retrievable in a structured format from the interface with no more than two interactions.

### **2.3.3 Performance**

1. PER-1: The system must identify fire or smoke within 2 seconds of its appearance in the camera's field of view.
2. PER-2: The system must refresh the room map with updated obstacle information within 1 second of recognizing a new obstacle.
3. PER-3: Fire suppression must commence within 5 seconds of detecting a fire, and small fires should be extinguished within 5 minutes.

### **2.3.4 Security**

1. SEC-1: The system will implement multi-factor authentication to ensure that only authorized personnel can access manual fire suppression controls.
2. SEC-2: All logs and event data must be encrypted while being transmitted to safeguard against tampering or unauthorized access.
3. • SEC-3: The system will automatically disable manual control if a security breach is detected during operation.

## 2.4 Domain Model

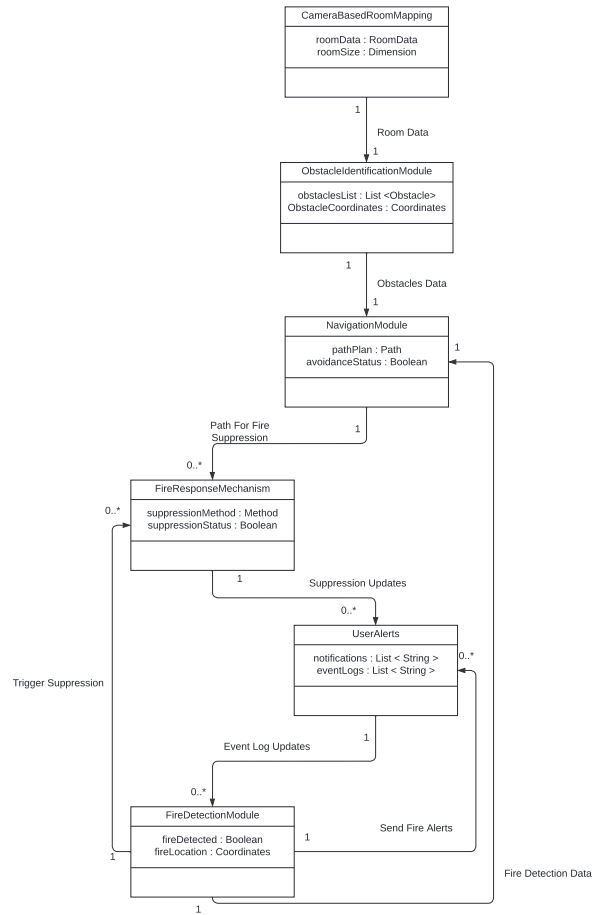


Figure 2.2: Domain Model

# Chapter 3

## System Overview

This project focuses on developing an object-oriented automated system that integrates real-time fire detection, room mapping, and obstacle identification. Utilizing advanced camera-based technology, the system ensures rapid alerts and response mechanisms for fire hazards, improving safety in critical environments like data centers and server rooms. Each feature of the system, such as room mapping and fire suppression, is designed as an object with its specific properties and methods, facilitating modularity and scalability. Key functionalities include accurate room layouts, efficient navigation, and automated fire suppression.

### 3.1 Architectural Design

The architectural design for the project consists of **two main modules**:

1. **Room Mapping and Obstacle Identification**
2. **Fire Detection and Response Mechanisms**

#### 3.1.1 Modular Structure and Relationships:

##### 1. **Camera-Based Room Mapping Module:**

- This module scans the environment using camera sensors to create a detailed map of the room. It also identifies and marks the locations of obstacles within the room to aid in navigation
- **Relationships:** The data from this module is sent to the Navigation System to ensure safe movement throughout the environment, helping to avoid obstacles.

#### 2. **Obstacle Identification Module:**

- This subsystem is closely linked to room mapping and detects both static and dynamic obstacles using the camera feed.
- **Relationships:** It works in conjunction with room mapping to provide real-time information to the Navigation Module, ensuring accurate positioning and movement.

#### 3. **Fire Detection Module:**

- This module identifies fire hazards in the environment through sensors, triggering immediate alarms and activating fire response protocols.
- **Relationships:** \* When a fire is detected, this module hands over control to the Fire Response System for further actions, such as suppression and alerts.

#### 4. **Fire Response Mechanism:**

- This encompasses both the automated fire extinguishing system and alert notifications to users. The module receives input from the fire detection module and initiates fire suppression actions.
- **Relationships:** It integrates directly with the Fire Detection Module, ensuring prompt action when a hazard is detected.

#### 5. **Navigation Module:**

- Utilizing data from room mapping and obstacle identification, this module calculates safe paths for necessary movements within the environment, including the option for manual override during fire emergencies.
- **Relationships:** It receives input from both the Camera-Based Room Mapping and Obstacle Identification modules.

#### 6. **User Interface Module** (Optional for external control):

- This allows users to manually interact with the system, view room layouts, see detected hazards, and initiate manual fire suppression if necessary.
- **Relationships:**\* It interfaces with all modules to provide users with real-time information and control options.

### 3.1.2 **High-Level Collaboration:**

The system features a modular architecture that enables each component to perform specific roles while working together to achieve overall functionality:

1. **Camera-Based Room Mapping Module:** This module starts the scanning process of the environment, creating a real-time map and identifying the room's layout.
2. **Obstacle Identification Module:** 2. This module works in tandem with the mapping module, enhancing the system's ability to detect and track obstacles within the room. This information is essential for safe navigation.
3. **Navigation Module:** It takes input from both the room mapping and obstacle identification modules to determine safe and efficient paths for movement. It can dynamically adjust the route if new obstacles are detected during navigation.
4. **Fire Detection Module:** This module continuously monitors for fire hazards using visual and thermal sensors. It alerts the navigation module of any detected fire, allowing the FireCar to redirect if necessary.
5. **Fire Response Mechanism:** Once the fire detection module confirms the presence of fire, this mechanism activates the appropriate fire suppression systems (such as CO2 or water mist) and triggers alerts. This module is vital for ensuring a quick response to fire hazards, enhancing safety.
6. **User Interface Module (Optional):** This allows the admin to monitor the system's status in real-time, manually control the FireCar if needed, and review alerts and logs related to fire incidents.

## 3.2 Design Models

- Activity Diagram

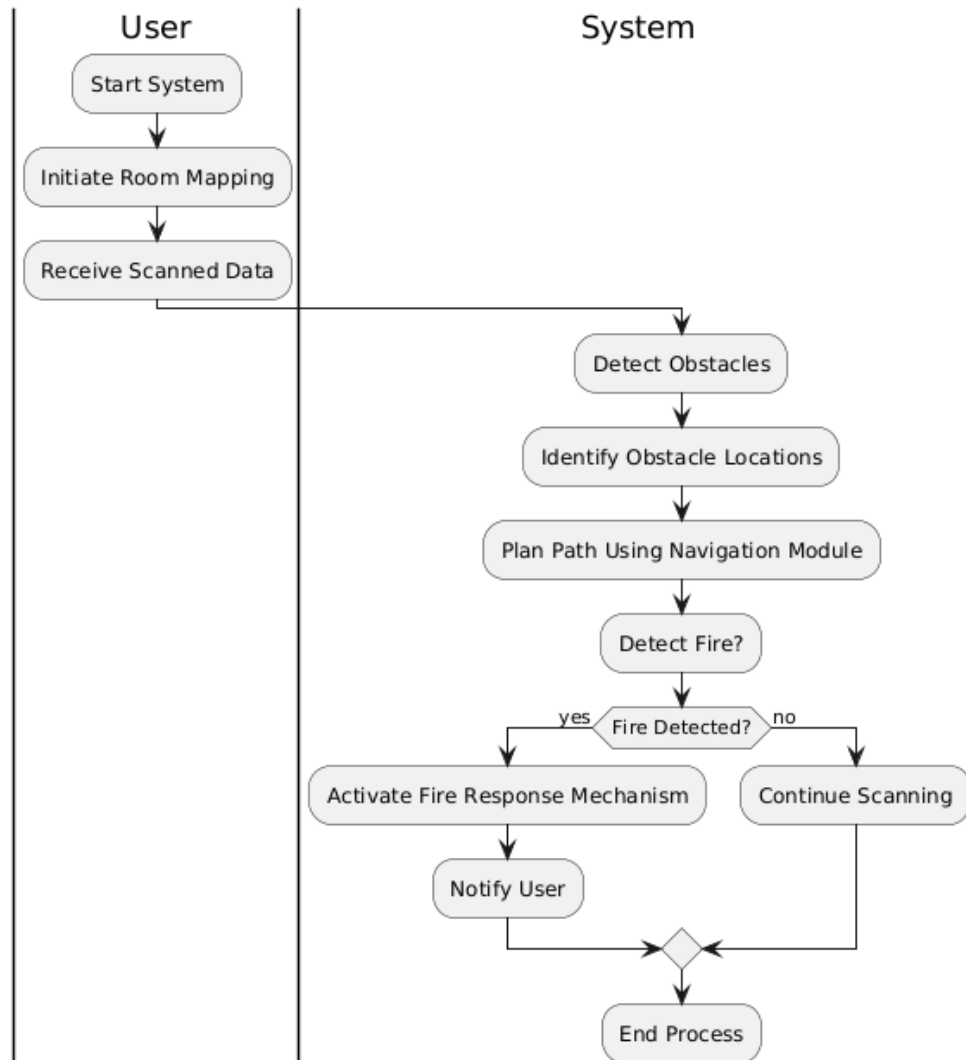


Figure 3.1: Activity Diagram

- Class Diagram

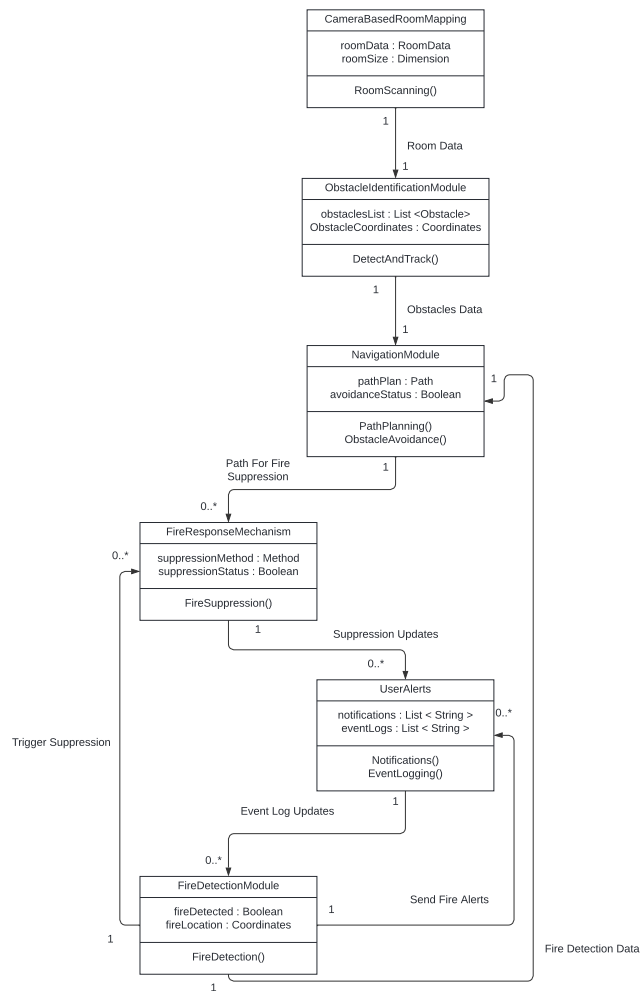


Figure 3.2: Class Diagram

- Class-level Sequence Diagram

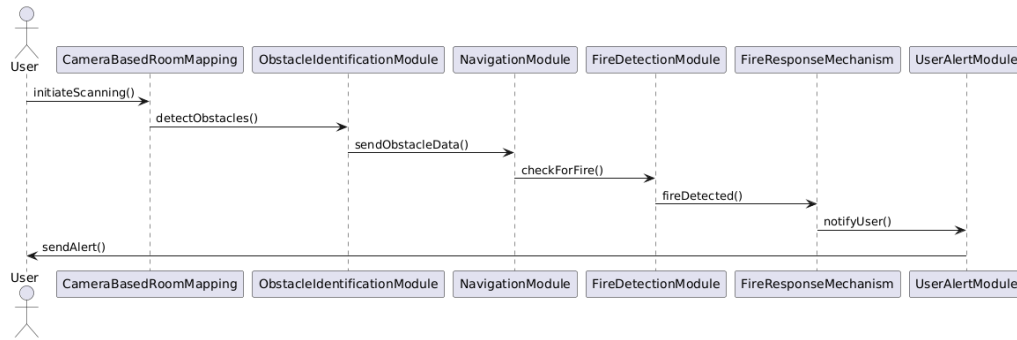


Figure 3.3: Class Level Sequence Diagram

- State Transition Diagram

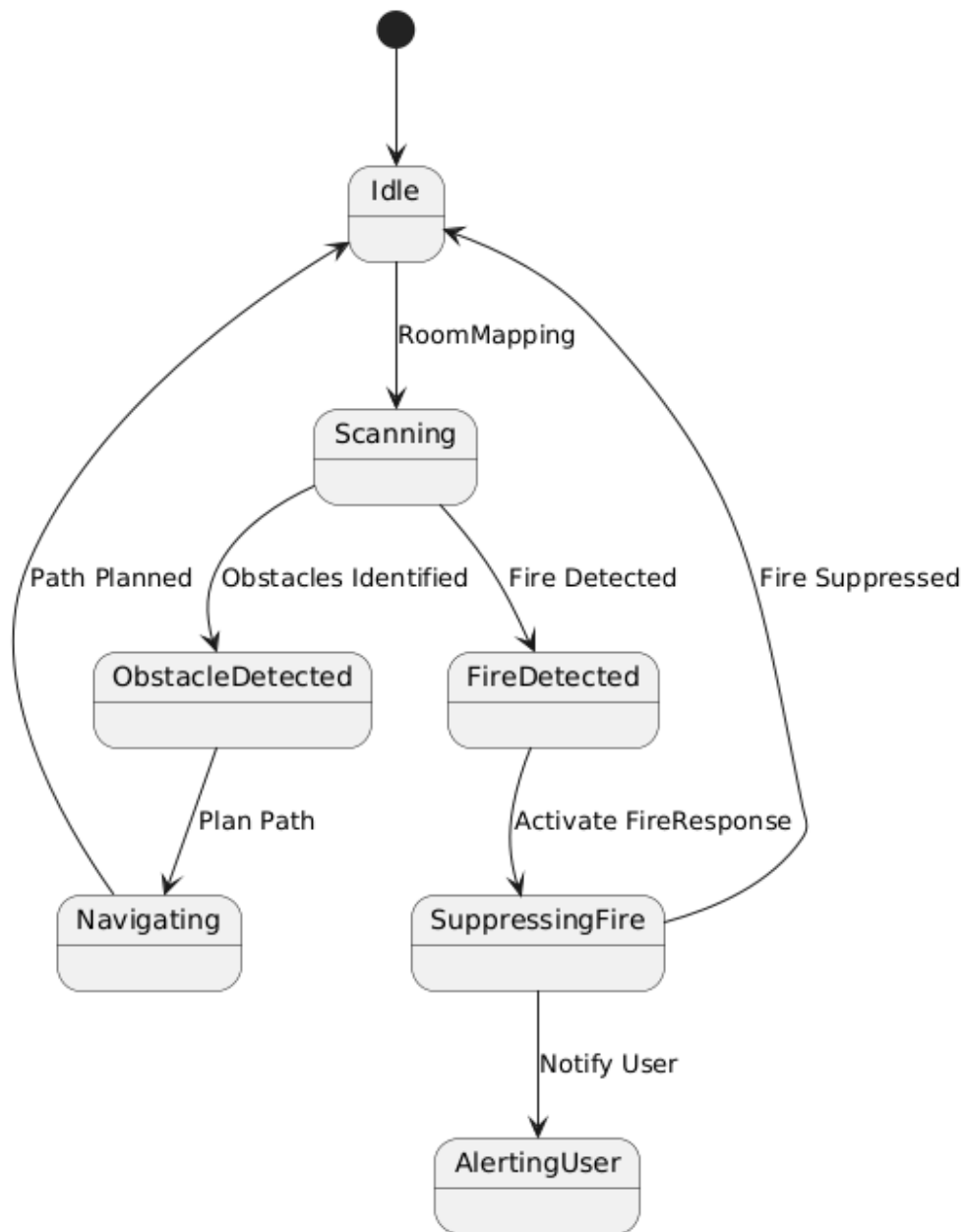


Figure 3.4: State Transition Diagram

## 3.3 Data Design

### 3.3.1 Real-time Data Capture

As previously mentioned, overhead cameras provide live video feeds that are crucial for creating maps, detecting fires, and spotting obstacles.

### 3.3.2 Data Structures

**Frame Data:** Each video frame or processed image can be stored in MongoDB as a document, which includes metadata (such as timestamp and image format) along with essential data (like fire detection status).

**Map and Environment Data:** The occupancy grid map generated from camera feeds can also be saved in a JSON-like format in MongoDB, allowing for quick access and updates when the environment changes. For example, each room's map can be saved as a document with fields for grid dimensions, obstacle locations, and fire spots.

**Fire and Obstacle Data:** MongoDB is ideal for storing objects like detected fires and obstacles, enabling you to maintain a collection of fire detection incidents, each represented by a document. This data can later be queried for reporting or used in real-time decision-making.

### 3.3.3 Data Processing

Processed frames can be indexed and stored in MongoDB, while maps and obstacles are updated in real-time as MongoDB documents.

### 3.3.4 Data Organization with MongoDB

**Collections:** MongoDB organizes data into collections, making it easy to separate different data types. For instance:

- A frames collection can hold data related to real-time images.
- A maps collection can store occupancy grid maps for each room.

- A fire detection collection can track fire incidents (including time, location, and severity).

Document Structure: Each real-time frame, map, or incident will be stored as a document, providing flexible and scalable data storage.



# **Bibliography**